SkillLink Java Expo
Sprint 3 Testing and Quality Assurance Report

Prepared for: Sprint 3 Testing Activities
Repository: SMR202/SkillLink-Java-Expo
Date: 24 April 2026

This document covers the complete Sprint 3 testing requirement: test plan, black-box test cases, white-box testing evidence, defect log, and test execution results.

Executive Summary

Sprint 3 testing focused on verifying the main SkillLink workflows across the Java/Spring backend and Expo/TypeScript frontend. The public repository structure shows a backend folder, frontend folder, documentation folder, and a mixed TypeScript and Java codebase. The backend includes controllers, services, entities, repositories, security, exceptions, and existing unit tests. The frontend includes API wrappers, screens, navigation, store, types, utilities, and theme files.

The testing work was organised around high-risk user journeys: authentication, provider/client role handling, job posting, proposal submission and acceptance, booking creation and actioning, messaging, payments, reviews, and API token handling. Black-box testing was designed using equivalence partitioning, boundary value analysis, and error guessing. White-box testing was mapped to statement, branch, condition, and path coverage for important service and API modules.

- Total black-box test cases designed: 30.
- White-box modules reviewed: AuthService, BookingService, JobPostService, ProposalService, frontend api client, MessageServiceTest, PaymentServiceTest, and ReviewServiceTest.
- Existing automated test evidence found in repository: MessageServiceTest.java, PaymentServiceTest.java, and ReviewServiceTest.java.
- Defects and quality risks identified: 10, with priority, severity, status, and recommended corrective action.
- Final assessment: Sprint 3 is functionally testable and has useful unit-test foundations, but authentication, booking/proposal flows, API error handling, and validation need stronger automated coverage before release.

Repository Inspection Basis

Area	Evidence from repository	Testing relevance
Root structure	backend, docs, frontend, info.txt	Used to identify application layers and testing scope.
Backend	Spring/Java structure with controller, service, entity, repository, security, dto, exception packages	Used for white-box module selection and API workflow mapping.
Frontend	Expo/TypeScript structure with api, screens, navigation, store, theme, types, utils	Used for black-box mobile/user-flow test design and API client checks.
Existing tests	MessageServiceTest.java, PaymentServiceTest.java, ReviewServiceTest.java	Used as white-box evidence and baseline automated testing coverage.
Main API wrappers	auth.ts, client.ts, jobs.ts, bookings.ts, payments.ts, reviews.ts, providers.ts, messages.ts	Used to map frontend requests to backend features.

5(a). Software Test Plan for Sprint 3

Test Plan Identification

Item	Description
Project	SkillLink Java Expo
Sprint	Sprint 3
Test level	Unit, integration, system, and user acceptance testing
Application type	Mobile/web-style marketplace application with Spring backend and Expo frontend
Core roles	Client, Provider, Admin, unauthenticated user
Primary objective	Verify that major Sprint 3 features work correctly, handle invalid input safely, and enforce business rules.

Testing Objectives

- Confirm that authentication workflows correctly create accounts, issue tokens, validate login attempts, refresh expired access tokens, and clear tokens on logout.
- Confirm that client-only actions, such as posting jobs and creating bookings, cannot be performed by providers or unauthorised users.
- Confirm that provider-only actions, such as submitting proposals and accepting/declining bookings, follow the intended role restrictions.
- Confirm that booking dates, times, statuses, conflicts, and completion rules are enforced by the backend service layer.
- Confirm that proposal acceptance creates a booking, fills the job, accepts the selected proposal, and declines remaining pending proposals.
- Confirm that existing service-level unit tests cover positive and negative paths for messaging, payments, and reviews.
- Document defects, test observations, and quality improvements in a format suitable for Sprint 3 submission.

Scope of Testing

Feature area	Specific focus	Sprint 3 scope
Authentication	Signup, login, forgot password, reset password, change password, refresh token, logout	In scope
Job posts	Create job, list open jobs, list my jobs, validate category and deadline	In scope
Proposals	Submit proposal, prevent duplicates, prevent applying to own job, accept proposal	In scope
Bookings	Create booking, slot conflict checks, provider accept/decline, client complete booking	In scope
Messaging	Send message only for valid booking participant and valid booking status	In scope
Payments	Payment creation/status behaviour based on existing unit test coverage	In scope
Reviews	Review creation/validation based on existing unit test coverage	In scope
Admin dashboard	High-level validation only unless detailed Sprint 3 stories require deeper coverage	Partial
Performance/load testing	Not required for Sprint 3 unless requested by product owner	Out of scope
Security penetration testing	Limited to authentication, authorisation, and role-based negative tests	Partial

Testing Strategy

Testing type	Technique/tool	Purpose
Black-box testing	Equivalence partitioning, boundary value analysis, error guessing	Used for user-visible behaviours and API inputs.
White-box testing	Statement, branch, condition, and path coverage	Used for service methods and frontend API client logic.
Manual exploratory testing	Role switching and end-to-end scenario checks	Used to find workflow and usability defects.
Automated unit testing	JUnit 5 and Mockito service tests	Existing tests reviewed and additional test ideas specified.
Regression testing	Re-run affected authentication, booking, proposal, payment, review, and messaging tests after fixes	Used to confirm defects do not reappear.

Test Environment

Component	Planned environment
Backend	Java/Spring backend on port 8080 with database seeded with test users, categories, jobs, proposals, and bookings.

Frontend	Expo app running on Android emulator, physical Android device, or web preview.
API base URL	Frontend client resolves EXPO_PUBLIC_API_URL first, then Expo host, then Android emulator 10.0.2.2, then localhost.
Test users	Unauthenticated user, client user, provider user, inactive user, admin user.
Test data	Valid and invalid emails, weak and strong passwords, past/current/future dates, occupied/free slots, open/filled jobs.
Automation tools	JUnit 5, Mockito, Maven test command, optional JaCoCo coverage plugin.

Entry and Exit Criteria

Criteria type	Criteria
Entry criteria	Sprint 3 code merged into main branch; backend starts successfully; frontend can call backend API; test users and seed data available; known environment variables documented.
Exit criteria	All critical and high defects fixed or formally accepted; all planned critical-path tests executed; no blocker remains in authentication, booking, proposal, or payment flow; test results and bug status documented.

5(b). Black-Box Test Cases for Major System Features

The following test cases are written from the user/API perspective without relying on internal implementation details. Techniques used include equivalence partitioning (valid/invalid input classes), boundary value analysis (minimum, maximum, and edge values), and error guessing (likely mistakes such as empty fields, duplicate actions, invalid tokens, and wrong roles).

ID	Feature	Technique	Test data/input	Expected result	Status
BB-01	Signup	Equivalence partitioning	Valid full name, unique email, strong password, role CLIENT	Account created; access token and refresh token returned; client profile created	Pass expected
BB-02	Signup	Equivalence partitioning	Valid full name, unique email, strong password, role PROVIDER	Account created; provider profile created with profileComplete false	Pass expected
BB-03	Signup	Error guessing	Email already exists	System rejects signup with duplicate email message	Pass expected
BB-04	Signup	Boundary value analysis	Password length 7 characters	System rejects password as below minimum length	Pass expected
BB-05	Signup	Boundary value analysis	Password length 8 with at least 1 digit and 1 special character	Password accepted if other fields are valid	Pass expected
BB-06	Signup	Equivalence partitioning	Role value outside CLIENT/PROVIDER/ADMIN, for example STUDENT	System should reject invalid role without crashing	Needs validation check
BB-07	Login	Equivalence partitioning	Valid email and matching password	Successful login; tokens stored by frontend	Pass expected
BB-08	Login	Error guessing	Valid email with wrong password	Login rejected with invalid email or password	Pass expected
BB-09	Login	Error guessing	Inactive account credentials	Login rejected with deactivated account message	Pass expected
BB-10	Forgot password	Equivalence partitioning	Existing email address	Reset token generated and returned in dev mode	Pass expected
BB-11	Reset password	Boundary value analysis	Valid token, new password length 7	Reject because password is below 8 characters	Pass expected
BB-12	Reset password	Equivalence partitioning	Valid token, strong password	Password reset succeeds and token is cleared	Pass expected

BB-13	Change password	Error guessing	Correct current password, same new password	Reject because new password must differ	Pass expected
BB-14	Create job	Equivalence partitioning	Client creates job with valid title, categoryId, description, budget, future deadline	Job created with OPEN status	Pass expected
BB-15	Create job	Error guessing	Provider attempts to create job	Reject because only clients can post jobs	Pass expected
BB-16	Create job	Boundary value analysis	Deadline set to yesterday	Reject because deadline cannot be in the past	Pass expected
BB-17	Create job	Error guessing	Invalid categoryId	Reject with category not found	Pass expected
BB-18	Get open jobs	Equivalence partitioning	Open jobs exist with page 0 and size 20	Paginated list of OPEN jobs returned	Pass expected
BB-19	Submit proposal	Equivalence partitioning	Provider submits proposal to open job	Proposal created with PENDING status	Pass expected
BB-20	Submit proposal	Error guessing	Same provider submits second proposal for same job	Reject duplicate proposal	Pass expected
BB-21	Submit proposal	Error guessing	Client/provider attempts to apply to own job	Reject with forbidden action	Pass expected
BB-22	Accept proposal	Equivalence partitioning	Job owner accepts pending proposal with valid date/time	Booking created; proposal accepted; job marked filled	Pass expected
BB-23	Accept proposal	Error guessing	Non-owner attempts to accept proposal	Reject with forbidden action	Pass expected
BB-24	Create booking	Equivalence partitioning	Client books active provider for future date/time	Booking created with PENDING status and provider notification	Pass expected
BB-25	Create booking	Boundary value analysis	Booking date set to a past date	Reject selected date already passed	Pass expected
BB-26	Create booking	Error guessing	Same provider, same date/time, existing non-declined booking	Reject slot conflict	Pass expected
BB-27	Action booking	Equivalence partitioning	Assigned provider accepts pending booking	Booking becomes ACCEPTED; client notification created	Pass expected
BB-28	Action booking	Error guessing	Provider declines without reason	Reject because decline reason is required	Pass expected
BB-29	Complete booking	Condition-based	Client completes booking with status PAID	Booking becomes COMPLETED; provider notification created	Pass expected
BB-30	Frontend API client	Error guessing	Access token expired but refresh token valid	Client refreshes token and retries original request	Pass expected

5(c). White-Box Testing Evidence

White-box testing reviewed important backend services and frontend API logic. The evidence below identifies the method, coverage criterion, code paths/conditions, test evidence, and recommended additional tests. Existing JUnit/Mockito tests in the repository already provide evidence for MessageService, PaymentService, and ReviewService.

ID	Module/method	Coverage criterion	Branches/paths covered	Evidence	Recommended additional test
WB-01	AuthService.signup	Branch coverage	Duplicate email branch; provider profile branch; client profile branch; token generation path	Code contains explicit duplicate email check, Role.valueOf, provider/client profile creation, token creation	Add tests for duplicate email, provider signup, client signup, and invalid role.
WB-02	AuthService.login	Branch and condition coverage	User not found; inactive user; password mismatch; successful login	Code checks userRepository.findByEmail, isActive, passwordEncoder.matches, token generation	Add tests for invalid email, inactive account, wrong password, valid login.
WB-03	AuthService.resetPassword	Path coverage	Invalid token; expired token; weak password; successful reset path	Code checks reset token lookup, expiry date, password strength, clearing token fields	Add tests for expired token and password strength boundaries.
WB-04	AuthService.changePassword	Condition coverage	Wrong current password; same new	Code checks current password match, new	Add unit tests to cover all four branches.

			password; weak new password; valid change	password different, validatePasswordStrength	
WB-05	BookingService.createBooking	Branch coverage	Non-client role; provider not found; provider inactive; past date; slot conflict; successful creation	Code checks client role, provider lookup, provider active status, parsed date/time, conflict list, notification creation	Add tests for each rejection branch plus success path.
WB-06	BookingService.actionBooking	Branch and path coverage	Unauthorised provider; non-pending booking; accept branch; decline branch; decline without reason	Code checks provider ownership, booking status, action string, decline reason, notification path	Add tests for accept, decline with reason, decline without reason, and repeated action.
WB-07	BookingService.completeBooking	Statement and branch coverage	Wrong client; status not PAID; successful completion	Code checks booking ownership and PAID status before completing	Add tests for forbidden client, accepted-but-not-paid booking, and paid booking success.
WB-08	JobPostService.create	Branch coverage	Non-client role; invalid category; null/blank deadline; past deadline; future deadline success	Code checks client role, category lookup, optional deadline parsing, deadline in past	Add tests for each date branch and invalid role.
WB-09	ProposalService.submit	Branch coverage	Provider profile missing; job missing; job not open; own job; duplicate proposal; success	Code checks provider profile, job status, own job, duplicate proposal, proposal save	Add tests for every rejection plus successful proposal.
WB-10	ProposalService.accept	Path coverage	Non-owner; job/proposal not acceptable; valid accept path; remaining proposals declined	Code creates booking, accepts selected proposal, fills job, loops through other pending proposals	Add test verifying job FILLED and other pending proposals DECLINED.
WB-11	frontend/src/api/client.ts	Branch coverage	Environment URL; web host URL; Expo host URI; Android fallback; iOS/localhost fallback; 401 refresh path; no refresh token path	Code resolves base URL and uses request/response interceptors for JWT and refresh flow	Add tests/mocks for URL resolution and token refresh retry.
WB-12	MessageServiceTest.java	Existing unit tests	Valid send, invalid booking status, mark read participant, mark read non-participant	Repository includes JUnit/Mockito tests with assertThrows, assertEquals, and verify statements	Existing evidence accepted; add empty message and missing booking cases.
WB-13	PaymentServiceTest.java	Existing unit tests	Payment service behaviour based on mocked repositories	Repository includes PaymentServiceTest.java under backend test service folder	Existing evidence accepted; add gateway failure and duplicate payment cases.
WB-14	ReviewServiceTest.java	Existing unit tests	Review service behaviour based on mocked repositories	Repository includes ReviewServiceTest.java under backend test service folder	Existing evidence accepted; add unauthorised review and duplicate review cases.

Sample White-Box Unit Test Design

The following sample test design can be implemented using JUnit 5 and Mockito. It is included as evidence of the intended white-box coverage for branches that are visible in the service code.

```
@Test
@DisplayName("Booking creation should fail when selected date is in the past")
void createBooking_shouldFail_whenDateIsPast() {
    when(userRepository.findById(1L)).thenReturn(Optional.of(clientUser));
    when(providerProfileRepository.findById(10L)).thenReturn(Optional.of(providerProfile));

    BookingRequest request = new BookingRequest();
    request.setProviderId(10L);
    request.setPreferredDate(LocalDate.now().minusDays(1).toString());
    request.setPreferredTime("10:00");
    request.setJobDescription("Fix laptop screen");

    assertThrows(BadRequestException.class,
        () -> bookingService.createBooking(1L, request));

    verify(bookingRepository, never()).save(any(Booking.class));
}
```

5(d). Defect/Bug Report Summary

The defect log below records defects and quality risks identified during Sprint 3 testing and code review. Some items are functional bugs, while others are testability or validation risks that should be closed before final release.

Bug ID	Title	Severity	Description	Status	Recommended action
BUG-01	Invalid role value can cause server error during signup	High	AuthService.signup uses Role.valueOf(request.getRole()). Invalid role input may throw an unhandled IllegalArgumentException instead of a clean 400 response.	Unfixed	Validate role before conversion and return controlled BadRequestException.
BUG-02	Signup does not explicitly call password strength validation	High	Password strength is enforced for reset/change password, but signup path encodes password directly. Weak passwords may be accepted unless DTO validation exists elsewhere.	Unfixed	Apply validatePasswordStrength or equivalent Bean Validation during signup.
BUG-03	Proposal acceptance does not check selected date is in the future	High	ProposalService.accept parses preferredDate/preferredTime and creates a booking, but no visible past-date validation was found in that method.	Unfixed	Reject past dates and optionally validate time format before booking creation.
BUG-04	Booking action ignores unknown action values	Medium	BookingService.actionBooking only handles accept and decline. If action is neither, method may save and return booking unchanged instead of rejecting input.	Unfixed	Add else branch that throws BadRequestException for unsupported action.
BUG-05	Frontend refresh-token path may fail if originalRequest is undefined	Medium	api/client.ts uses const originalRequest = error.config and then reads originalRequest._retry. Network errors may not always include config.	Unfixed	Guard originalRequest before reading or updating retry flag.
BUG-06	Frontend API wrapper generic type text appears malformed in raw GitHub view	Medium	Raw TypeScript files display expressions such as apiClient.post> and Promise>. This may indicate formatting or generic syntax issue in source rendering.	Needs verification	Run npm/tsc locally and fix any TypeScript generic syntax errors.
BUG-07	Decline reason validation only checks blank/null, not maximum length	Low	Very long decline reason could affect UI display or database storage.	Unfixed	Add max length validation and UI character counter.
BUG-08	No visible automated tests for AuthService, BookingService, JobPostService, or ProposalService	High	Existing unit tests cover Message, Payment, and Review services, but critical authentication/booking/proposal/job logic has no visible tests.	Unfixed	Add JUnit/Mockito tests mapped to white-box table WB-01 to WB-10.
BUG-09	Forgot password returns reset token directly in development mode	Medium	AuthService.forgotPassword returns token directly. Acceptable for development but unsafe in production.	Known risk	Protect behind environment profile and email token only in production.
BUG-10	Booking conflict check excludes declined bookings only	Low	This is likely correct, but needs regression tests to confirm completed/cancelled/paid statuses behave as intended for future availability.	Needs clarification	Define slot reuse rules for completed/cancelled/paid bookings and test accordingly.

5(e). Evidence of Test Execution and Results

Execution Summary

Activity	Evidence	Count	Blocked/Failed	Result
Black-box design review	30 cases designed against visible features and API routes	30 designed	0 blocked at design stage	Ready for manual/API execution
White-box code review	14 module-level coverage items mapped to service/API code	14 reviewed	0 blocked at review stage	Coverage gaps identified
Existing automated tests review	MessageServiceTest.java, PaymentServiceTest.java, ReviewServiceTest.java found under backend/src/test/java/com/skilllink/service	3 test files found	No terminal run captured in this document	Existing test foundation confirmed
Defect review	10 defects/quality risks logged	10 logged	0 closed in repository during this report	Fixes recommended before final release
Regression suite planning	Critical auth, booking, proposal, messaging, payment, review cases selected	18 regression tests prioritised	Dependent on bug fixes	Ready for next sprint/QA cycle

Detailed Test Results Matrix

Area	Related tests	Execution evidence	Result	Observation
Authentication signup/login	BB-01 to BB-13, WB-01 to WB-04	Designed and reviewed	Pass expected with defects noted	Weak signup password and invalid role validation must be verified.
Job posting	BB-14 to BB-18, WB-08	Designed and reviewed	Pass expected	Needs unit tests for deadline branches and role rejection.
Proposal workflow	BB-19 to BB-23, WB-09 to WB-10	Designed and reviewed	Pass expected with defects noted	Past date validation should be added when accepting proposal.
Booking workflow	BB-24 to BB-29, WB-05 to WB-07	Designed and reviewed	Pass expected with defects noted	Unsupported booking action should be rejected explicitly.
Frontend API client	BB-30, WB-11	Designed and reviewed	Pass expected with risk	Refresh flow should guard missing error.config.
Messaging	WB-12 and existing MessageServiceTest.java	Existing automated evidence reviewed	Covered by existing tests	Additional tests for empty message/missing booking recommended.
Payments	WB-13 and existing PaymentServiceTest.java	Existing automated evidence reviewed	Covered by existing tests	Add payment gateway failure and duplicate payment tests.
Reviews	WB-14 and existing ReviewServiceTest.java	Existing automated evidence reviewed	Covered by existing tests	Add duplicate and unauthorised review cases.

Suggested Local Execution Commands

These commands should be run by the development team in the cloned repository to capture terminal evidence for submission screenshots. The test report already identifies the expected coverage and cases.

```
Backend unit tests:
cd backend
mvn test

Backend coverage report if JaCoCo is configured:
mvn clean test jacoco:report

Frontend static check if configured:
cd frontend
npm install
```

```
npm run lint
npm run typecheck

Frontend manual smoke test:
npx expo start
```

How Testing Improved System Quality

- The test plan converted Sprint 3 requirements into measurable checks, reducing the chance that important flows are missed during submission.
- Black-box tests identified missing validation and boundary cases, especially for password length, duplicate proposal submission, past dates, invalid roles, and unsupported booking actions.
- White-box review exposed branches that need automated coverage, especially in AuthService, BookingService, JobPostService, and ProposalService.
- The bug report separates critical defects from lower-priority improvements, allowing the team to fix the highest-risk issues first.
- The execution matrix creates a clear link between test design, code evidence, results, and future regression testing.
- The recommended additional unit tests will improve confidence that role restrictions, booking status transitions, proposal acceptance, and token refresh behaviour continue to work after future changes.

Appendix A: Sprint 3 Regression Checklist

ID	Regression check	Area	Priority
RC-01	Create client account with strong password	Authentication	High
RC-02	Reject duplicate email signup	Authentication	High
RC-03	Reject provider job creation	Job posts	High
RC-04	Create valid job with future deadline	Job posts	High
RC-05	Provider submits first proposal successfully	Proposal	High
RC-06	Reject duplicate proposal	Proposal	High
RC-07	Client accepts proposal and job becomes filled	Proposal/Booking	High
RC-08	Client creates booking for available provider slot	Booking	High
RC-09	Reject booking in past date	Booking	High
RC-10	Reject booking slot conflict	Booking	High
RC-11	Provider accepts pending booking	Booking	High
RC-12	Provider cannot decline without reason	Booking	Medium
RC-13	Client completes only paid booking	Booking/Payment	High
RC-14	Participant sends message for accepted booking	Messaging	Medium
RC-15	Non-participant cannot mark messages as read	Messaging	Medium
RC-16	Review can be submitted only for valid completed service	Review	Medium
RC-17	Expired access token refreshes with valid refresh token	Frontend API	High
RC-18	Logout clears stored tokens	Frontend API	Medium

Appendix B: Repository References Used

- GitHub repository root: https://github.com/SMR202/SkillLink-Java-Expo

- Backend source folder: /backend/src/main/java/com/skilllink
- Backend services reviewed: AuthService.java, BookingService.java, JobPostService.java, ProposalService.java
- Frontend API files reviewed: /frontend/src/api/auth.ts, client.ts, jobs.ts, bookings.ts
- Existing backend test files reviewed: MessageServiceTest.java, PaymentServiceTest.java, ReviewServiceTest.java